

HOP-PAAL: A FORMAL FROGGER

Formal Methods for Software Engineering Project

MICHAEL ALIBEAJ (michael.alibeaj@mail.polimi.it)

ADEM SHEHI (adem.shehi@mail.polimi.it)

July 23, 2026

ABSTRACT We model the Atari 2600 game Frogger as a network of timed automata in Uppaal. In the symbolic version we prove seven TCTL properties, including deadlock freedom, reachability of specific bays and scores, and a death by a submerged turtle. In the stochastic extension we show that a goal biased safe hop heuristic raises the probability of completing the first level from essentially zero to a statistically significant fraction, while also exposing long game hazards that a baseline random walk never survives long enough to encounter.

CONTENTS

1	Introduction	3
1.1	Definitions	3
1.2	Software and machines used	3
2	Model description	3
2.1	Entities modeling (symbolic version)	3
2.1.1	Map	3
2.1.2	Movement model: per row offset	4
2.1.3	Lanes	4
2.1.4	Diving turtles	4
2.1.5	Frogger	4
2.1.6	GameManager	5
2.2	Communication and atomicity	5
2.3	Enrichment of the symbolic model with stochastic features	5
3	Verification results	6
3.1	Symbolic model verification	6
3.1.1	Queries on the environment only (I, II)	6
3.1.2	Reachability queries on the full system (III to VII)	6
3.2	Stochastic model verification	7
4	Analysis of selected configurations	9
5	Conclusions	10
A	Design alternatives	11
A.1	Per object coordinates versus per row offset	11
A.2	Dedicated map template vs implicit getCellType	11
A.3	Per turtle diving clocks versus a single DivingCycle	11
A.4	GameOver self loop versus the two system toggle	11
A.5	Visible width wraparound versus the ring of 20 cells	11
A.6	Local clocks versus a single globalClock	11

LIST OF FIGURES

Figure 1	Left: Query I probability estimates with 95% confidence intervals. Right: sample score trajectories from Query II; baseline runs stagnate at low scores, while heuristic runs can climb past the 200 point threshold.	7
Figure 2	Distribution of final scores over 100 simulation runs. The baseline never reaches the 200 point level 1 threshold, while the goal biased heuristic produces a right skewed distribution with 28 runs clearing the threshold.	9
Figure 3	Left and middle: survival time distributions over 100 simulation runs per strategy; solid blue lines mark the mean, dashed orange lines the median. Right: stacked death cause shares. The baseline only experiences car and water deaths, while the heuristic additionally encounters timeout, submerged turtle, and deadly cell deaths.	10

LIST OF TABLES

Table 1	Software used	3
Table 2	Machine specifications	3
Table 3	Overview of queries on the environment only	6
Table 4	Overview of reachability queries on the full system	6
Table 5	Baseline versus goal biased heuristic, simulation metrics.	9

1 INTRODUCTION

1.1 Definitions

- **Cell Types:** five classes returned by the map function — **safe**, **road**, **water**, **home** (the five bays), and **deadly** (gaps between bays). Road rows carry vehicles, water rows are deadly unless the frog is on a floating object.
- **Bay:** one of the five top row home cells. Filling all five completes a level.
- **Time Unit:** the model's discrete time step. 10 t.u. = 1 s.
- **Effective Column / Wraparound Ring:** every object in the same row shares a single offset. Its horizontal position is derived by wrapping on a ring of 20 cells (18 visible columns plus a buffer of 2 off-screen cells), so objects carried offscreen re-enter on the opposite side.
- **Carry:** when Frogger stands on a log or turtle, it moves horizontally with that object.
- **Diving turtle:** a turtle whose visibility follows a global cycle (Emerged → Diving → Submerged → Emerging). A frog riding one when it submerges dies.
- **Observer Flag:** a boolean variable that records an event for verification but never influences game logic.
- **Urgent Broadcast Channel:** a synchronization primitive that forces participating automata to act at the same instant. `frog_died` and `frog_home` carry death/home events between Frogger and GameManager.
- **Round Timeout:** the time budget for each attempt to reach the top row. Exceeding it kills the frog.

1.2 Software and machines used

Usage	Software	Version
Modeling, simulation and verification	Uppaal	5.0.0
Report writing	Overleaf (pdfLaTeX)	n/a

Table 1: Software used

Each query was verified on the following machine:

CPU	RAM	OS
AMD Ryzen 7 8845HS	16 GB	Windows 11

Table 2: Machine specifications

2 MODEL DESCRIPTION

We modeled both the symbolic and the stochastic version of Frogger. We first modeled the symbolic version and then modified it to satisfy the stochastic requirements. This section explains both versions and how the stochastic one was obtained from the symbolic one.

2.1 Entities modeling (symbolic version)

2.1.1 Map

We keep the map implicit rather than as a separate template or matrix, in order to add no extra state for the verifier to explore. The grid is 18×13 and cell types are returned by the pure function `getCellType(x, y)`: rows 0 and 6 are **SAFE**, rows 1 to 5 are **ROAD**, rows 7 to 11 are **WATER**, and row 12 holds the five **HOME** bays (columns with $x \bmod 4 \in \{0, 1\}$) interleaved with **DEADLY** cells. Row band bounds are named constants (`SAFE_ROW_LOW`, `ROAD_ROW_MIN`, ...) to avoid scattered numeric literals.

2.1.2 Movement model: per row offset

The central modeling decision is to minimize the number of clocks and dynamic variables: instead of giving every object its own mutable coordinate and its own clock, we share a single per row offset among all objects in the same row. Each object is defined by an immutable pair $(\text{baseX}, \text{row})$ plus a size; the only mutable horizontal state is $\text{row_offset}[\text{row}]$. The current horizontal position is derived,

$$\text{effective_x} = (\text{baseX} + \text{row_offset}[\text{row}] + \text{VIRTUAL_W}) \bmod \text{VIRTUAL_W},$$

with $\text{VIRTUAL_W} = 20$ ($\text{COLS} = 18$ plus a buffer of 2 off screen cells). One lane automaton per moving row advances $\text{row_offset}[\text{row}]$ by ± 1 at each shift, so every object in that row moves together. This collapses what would be N independent clocks (one per object) into one clock per lane, and makes lane membership a static fact the verifier can exploit (see Appendix A.1). The cost is a less object oriented model and the need to reason about the off screen buffer globally.

2.1.3 Lanes

Because all objects in a row move in lockstep, one lane automaton per row is sufficient. We split road and river behavior into two templates, $\text{VehicleLane}(\text{row})$ and $\text{FloatingLane}(\text{row})$, so that collision checking (road) and frog-carry logic (river) remain in separate automata; this keeps each template's edges and invariants small and avoids mixing unrelated concerns. Each lane carries its own local clock t , which means rows advance independently and the verifier can handle their timing separately rather than coupling every moving object to a single global clock. The shift fires when $t == \text{row_speed}[\text{row}]$, increments the shared $\text{row_offset}[\text{row}]$ by $\text{row_dir}[\text{row}]$, and resets t . The speed and direction arrays match the spec tables: row 1 is the fast orange car (3 t.u.), row 2 the slow bulldozer (10 t.u.), and so on; river rows alternate directions as required.

2.1.4 Diving turtles

We avoid five separate clocks by letting all diving turtles share a single global cycle. A single global DivingCycle automaton owns the state globalDivState , cycling $\text{Emerged}(10) \rightarrow \text{Diving}(5) \rightarrow \text{Submerged}(5) \rightarrow \text{Emerging}(10)$ with durations stored in $\text{div_duration}[4]$. On entering Submerged , $\text{submergeFrogger}()$ scans the diving turtle set and kills a frog that is riding one. The spec gives one cycle with fixed durations and a common initial state for all diving turtles, so a shared global cycle is the most faithful interpretation and keeps the model small; the trade off is that turtles cannot dive out of phase (see Appendix A.3).

2.1.5 Frogger

The symbolic Frogger is deliberately minimal: it moves only upward, which is the smallest movement model that still exercises road crossing, river carry, bay entry, and every death mechanism required by the queries. We deliberately omitted left/right movement by the player and let floating objects provide all horizontal displacement, which exercises off-screen carry and re-entry without adding extra move logic.

A start wait $\text{FROG_WAIT} = 20$ (2 s) precedes the first move, matching the spec countdown. $\text{FROG_MIN_MOVE} = 1$ is a Zeno guard (Assumption 2): it guarantees at least one time unit between moves, so the symbolic model cannot take infinitely many hops in finite time. A roundClock bounded by $\text{currentTimeout}() = \max(1, 300 - 20(\text{level} - 1))$ kills the frog on timeout; the max-with-1 keeps the invariant non-trivial even at high levels. When the frog stands on a log or turtle, frogX follows that object's effective column, wrapping on the ring of 20 cells; $\text{carryFroggerIfNeeded}()$ kills the frog only if it re-enters the visible portion on a non-safe cell, while the two-cell off-screen buffer itself is safe.

For the reachability queries we added four boolean observer flags, all set by $\text{updateObservationFlags}()$ and strictly read-only. We chose observer flags rather than writing the conditions directly in the query formulas because the flags (i) record the exact instant the event happens and (ii) cannot influence the movement, scoring, or game state they observe. The flags are:

- reached_bay_0 : entered the leftmost home bay.
- $\text{reached_bay_2_with_17}$: entered the central bay with score exactly 17.
- $\text{reached_row_6_after_two_deaths}$: returned to the mid safe row after at least two deaths.
- $\text{died_by_submerged_turtle}$: killed by a diving turtle transitioning to submerged.

2.1.6 GameManager

We centralized `lives`, `score`, `level`, and the five `bayOccupied` flags in one `GameManager` automaton because these are global rules that do not naturally belong to any single environmental agent. Keeping them separate from the frog and from the lanes avoids duplicating state updates across templates and makes the scoring and level-transition logic easy to inspect. The manager synchronizes with the frog over two urgent broadcast channels, `frog_died` and `frog_home`; urgency guarantees that a death or home event is handled in the same instant it is detected, which prevents races where the frog could move again before the manager has finished processing the event. Broadcast semantics are also convenient here because they let the frog fire `frog_died!` even after the manager has reached `GameOver`, avoiding deadlock at game end.

Death is processed by `processDeath()`, which decrements `lives` and respawns the frog only when `lives > 0`. We added this guard after noticing that, without it, the frog was reset to the starting cell on the final death while the manager simultaneously entered `GameOver`, so it could continue scoring even though the game had already ended.

2.2 Communication and atomicity

Frogger and `GameManager` communicate over the urgent broadcast channels `frog_died` and `frog_home`. We use broadcast rather than handshake channels because the frog may need to signal a death after the manager has already reached `GameOver`; a handshake would block and create deadlock. Collision checking and frog-carry evaluation use committed locations, so the read of the environment and the frog's position update happen instantaneously and atomically; this prevents another lane from shifting between the moment a death is detected and the moment it is recorded.

2.3 Enrichment of the symbolic model with stochastic features

We built the stochastic version by reusing the entire symbolic environment (lanes, diving cycle, carry, `GameManager`) and changing only the places where the stochastic spec adds requirements. Reusing the environment means the SMC experiments isolate the effect of the movement strategy: any difference between the baseline and the heuristic comes from Frogger's decisions, not from different vehicle or river behavior.

The changes are concentrated in three places:

- **Movement.** A probabilistic branchpoint gives the four cardinal directions weights 5/10 (up), 1/10 (down), 2/10 (left), 2/10 (right). Only an up move scores +1, so the score still measures forward progress. We use a single shared `evaluateLanding()` for all directions so that death, carry, and bay handling are implemented once and behave identically regardless of the direction that triggered the hop.
- **Forced move.** Once the initial 2 s countdown elapses, the `Playing` invariant `moveClock <= FROG_FORCE_MOVE` removes the idle self-loop, so the frog must hop every time unit. This matches the spec requirement that Frogger "attempts to move frequently enough," and it prevents the SMC runs from containing arbitrarily long waiting sequences.
- **Exponential trucks.** Trucks on row 5 use a dedicated `TruckLane(row)` template with `exponentialRate 1:row_speed[row]` ($\lambda = 1/\text{speed}$), so the delay between consecutive truck moves follows an exponential distribution. Non-truck rows remain deterministic (`VehicleLane`) because the spec only requires randomness for trucks.
- **Strategy switch.** `const bool SMART_HEURISTIC` selects between the baseline random walk (`false`, used for the first SMC query) and a goal-biased safe-hop policy (`true`). In the heuristic mode `canMoveTo()` refuses unsafe cells (vehicles, open water, deadly cells), and any refused non-up hop falls back to an up hop. The probabilistic weights stay identical in both modes so that any performance difference is caused by the policy, not by a reweighted distribution.

3 VERIFICATION RESULTS

3.1 Symbolic model verification

The system declaration contains two alternative system blocks (one commented): a block with only the environment (lanes + DivingCycle, no Frogger/GameManager) used for the safety queries I to II, and a block with the full system used for the reachability queries III to VII.

We split the verification into two phases because the Uppaal deadlock keyword is global: a terminal GameOver location in the full system would be reported as deadlock even though the game has legitimately ended. Queries I to II therefore run on the environment alone, proving that the moving world never gets truly stuck. Queries III to VII run on the full system and prove reachability of specific game situations.

3.1.1 Queries on the environment only (I, II)

Uppaal reports elapsed verification time and resident/virtual memory peaks for each query. The two results are summarized in Table 3.

	Verified property	Result [T/F]	Elapsed time [s]	Memory peak (resident / virtual)
I	$A[] \neg \text{deadlock}$	T	11.588	148 072 / 321 624
II	$A[] \text{allObjectsStayInRows()} \wedge \text{allRowsHaveValidOffsets()}$	T	12.356	160 228 / 346 552

Table 3: Overview of queries on the environment only

Query I verifies that the environment never reaches a deadlock state.

$$A[] \neg \text{deadlock}$$

This query asks whether there is any reachable state in which no transition is enabled and no delay is possible. The result is true because every lane always has either a pending shift (when its clock reaches the row speed) or a legal time delay (while the clock is below it), and the DivingCycle likewise always has an outgoing transition or can delay. Thus no environment state is ever in true deadlock.

Query II verifies that every moving object stays in its own row and that every row offset remains a valid index.

$$A[] \text{allObjectsStayInRows()} \wedge \text{allRowsHaveValidOffsets()}$$

This is a structural sanity check for the per row offset design. `allObjectsStayInRows()` checks that vehicles occupy rows 1 to 5, floating objects rows 7 to 11, and diving turtles rows 7 and 10, i.e. that the immutable row field is never violated. `allRowsHaveValidOffsets()` checks that every offset stays within the ring of 20 cells. Because offsets are only changed by ± 1 and immediately wrapped modulo `VIRTUAL_W`, both properties hold in every reachable state.

3.1.2 Reachability queries on the full system (III to VII)

The five reachability queries use observer flags: they capture the exact instant an event occurs and do not alter game mechanics. The results are summarized in Table 4; their cost varies sharply depending on how long or constrained the witnessing path must be.

	Verified property	Result [T/F]	Elapsed time [s]	Memory peak (resident / virtual)
III	$E\langle \rangle \text{reached_bay_0}$	T	1.731	64 216 / 153 224
IV	$E\langle \rangle \text{reached_bay_2_with_17}$	T	1.780	64 784 / 154 308
V	$E\langle \rangle \text{reached_row_6_after_two_deaths}$	T	58.436	425 884 / 854 920
VI	$E\langle \rangle (\text{globalClock} \geq 100 \wedge \text{lives} == 5 \wedge \text{isOnLog}(\text{frogX}, \text{frogY}))$	T	552.262	1 811 452 / 3 651 540
VII	$E\langle \rangle \text{died_by_submerged_turtle}$	T	3.798	1 811 488 / 3 651 640

Table 4: Overview of reachability queries on the full system

Query III, $E\langle \rangle \text{reached_bay_0}$, is a short reachability property: the frog must cross the road, ride a log or turtle, and land in the leftmost bay. Query IV, $E\langle \rangle \text{reached_bay_2_with_17}$, adds an exact-score constraint: 12 upward steps plus the 5-point bay bonus must total 17 with no intermediate deaths or bonuses. Both are cheap because the witnessing path is short and the state space near the start is small.

Query V, $E\langle \rangle \text{reached_row_6_after_two_deaths}$, is already much more expensive. The verifier must explore the interleavings of two deaths, respawns, and safe returns before the frog can reach the mid safe row. Query VI, $E\langle \rangle (\text{globalClock} \geq 100 \wedge \text{lives} == 5 \wedge \text{isOnLog}(\text{frogX}, \text{frogY}))$, is the hardest of all: it forces a 10-second execution with no life lost while the frog is specifically on a log. The 1.8 GB memory peak reflects the large set of safe paths through traffic and water that satisfy all three conjuncts.

Query VII, $E\langle \rangle \text{died_by_submerged_turtle}$, is cheap again because the diving cycle is deterministic once the frog is positioned on a diving turtle. Its memory peak is essentially the same as Query VI because both queries explore the same full-system state space, but Query VII finds a witness almost immediately while Query VI must search deep into the tree.

3.2 Stochastic model verification

Exact symbolic model checking is no longer possible once the model contains probabilistic branchpoints: the 4-direction movement, the exponential truck, and the forced move create an uncountable family of stochastic executions. We therefore use Uppaal’s Statistical Model Checking (SMC) engine, which samples runs and estimates probabilities and distributions rather than exploring the state space exhaustively. We verified two SMC queries, each under both strategies (baseline random walk and goal biased safe hop). We used Uppaal’s default SMC statistical parameters ($\epsilon = 0.05$, confidence 0.95)¹ The Pr query uses adaptive (sequential) sampling, so the run count is not fixed. Our runs stopped at 72 for the baseline and 375 for the heuristic, while simulate uses the fixed $N = 100$.

Query I estimates the probability of reaching the first level with score ≥ 200 within 1000 time units:

$$\text{Pr}[\leq 1000](\langle \rangle \text{first_level_with_200})$$

The observer flag `first_level_with_200` is set exactly once, at the instant level 1 is completed, and only if the score (including the +100 level bonus) is already ≥ 200 . This avoids the ambiguity of a sticky level flag combined with a monotonically increasing score. The baseline probability is ≈ 0 (0/72, upper CI ≤ 0.0499), while the heuristic reaches 0.323 (121/375, 95% CI [0.276, 0.373]). Figure 1 (left) visualizes the two estimates.

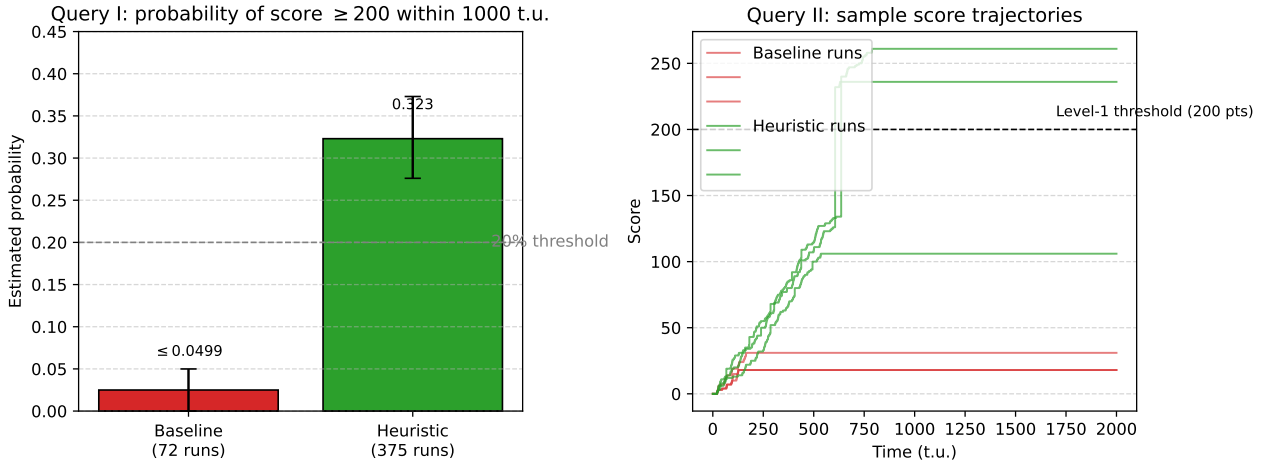


Figure 1: Left: Query I probability estimates with 95% confidence intervals. Right: sample score trajectories from Query II; baseline runs stagnate at low scores, while heuristic runs can climb past the 200 point threshold.

Query II is a fixed-N simulation of the game trajectory:

```
simulate[ $\leq 2000$ ; 100] {score, lives, level, deaths_by_car, deaths_by_water, levels_completed}
```

The six variables capture complementary aspects of each run: score measures forward progress (only up moves add points); lives tracks how long the frog survives and when the game ends; level shows how far into the multi-level structure a run progresses; deaths_by_car and deaths_by_water split the two main external hazards; and levels_completed records how many full levels were finished. We compare the two

¹ Default values reported by the Uppaal SMC options dialog; we did not override them.

strategies with identical move weights (5/1/2/2 for up/down/left/right), so any difference is attributable to the goal biased policy and not to a reweighted distribution.

Across the 100 baseline runs no frog completes level 1 (mean final score 21.4, survival ≈ 143 t.u.), whereas the heuristic completes level 1 in 28/100 runs and reaches level 4 in one run (mean final score 149.8, survival ≈ 587 t.u.). The detailed metric comparison and the simulation plots are reported in Section 4.

4 ANALYSIS OF SELECTED CONFIGURATIONS

For the stochastic Query II the spec asks us to analyze at least two interesting aspects of the game under the alternative strategy. We focus on two that the spec explicitly suggests: (1) progress, measured by completed levels and points scored, and (2) hazards, measured by survival time and the mix of death causes. Table 5 summarizes the metrics we use for these two aspects.

Metric	Baseline	Heuristic
Complete level 1	0/100	28/100
Max level reached	1	4
Final score mean / median / max	21.4 / 20 / 36	149.8 / 115 / 685
Survival (lives==0) [t.u.]	143	587 ($\sim 4.1\times$)
Deaths car / water / other	80% / 20% / 0%	61% / 25% / 14%
Score rate [up moves / t.u.]	0.149	0.255

Table 5: Baseline versus goal biased heuristic, simulation metrics.

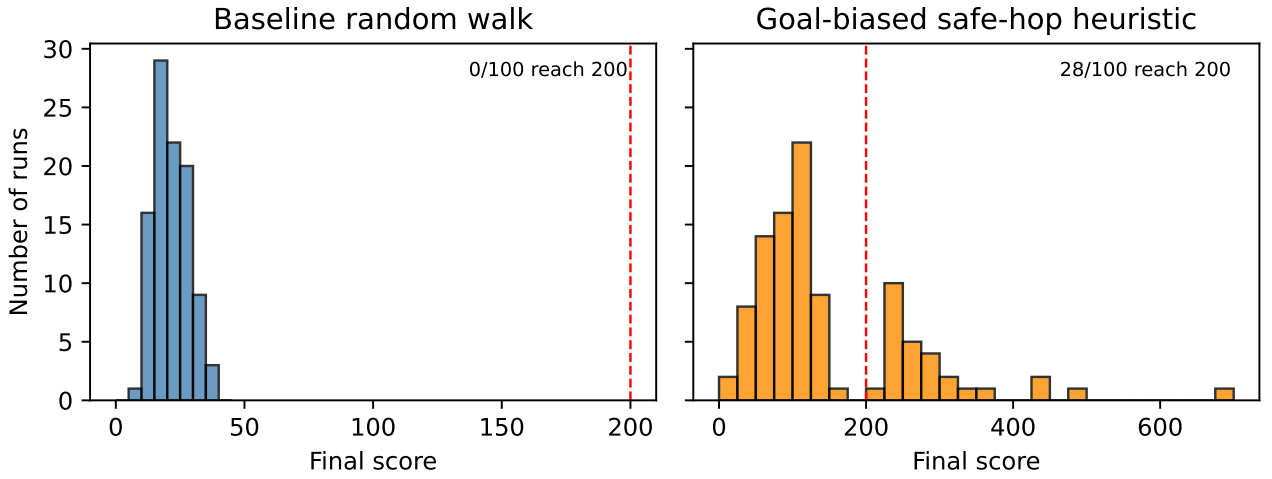


Figure 2: Distribution of final scores over 100 simulation runs. The baseline never reaches the 200 point level 1 threshold, while the goal biased heuristic produces a right skewed distribution with 28 runs clearing the threshold.

The empirical score distributions are shown in Figure 2. Under the heuristic, level 1 completion rises from 0% to 28% and survival more than quadruples ($143 \rightarrow 587$ t.u.). The reason is simple: the baseline random walk frequently hops sideways or down into unsafe cells or dead-end positions, while the heuristic refuses those hops and falls back to an up hop. Because the move weights are identical in both strategies (5/1/2/2), the difference is caused by the policy alone. The frog wastes fewer moves, reaches the water rows sooner, and rides floating objects more often; the score rate ($0.149 \rightarrow 0.255$ up moves/t.u.) reflects this.

Figure 3 visualises both the survival improvement and the change in death cause mix. The left panels show that the baseline is a narrow early peak (all 100 runs die before 200 t.u., mean 143.2), while the heuristic is a broad right skewed distribution centred around 500 to 600 t.u. (mean 586.9) with a long tail past 1700 t.u. The right panel explains why: the baseline only dies from car or water collisions (80%/20%), because it never survives long enough to see anything else. The heuristic survives long enough for long game hazards (timeout, submerged turtle, deadly cell) to account for 14% of deaths. These are not failures of the policy; they are the natural risks of a longer game.

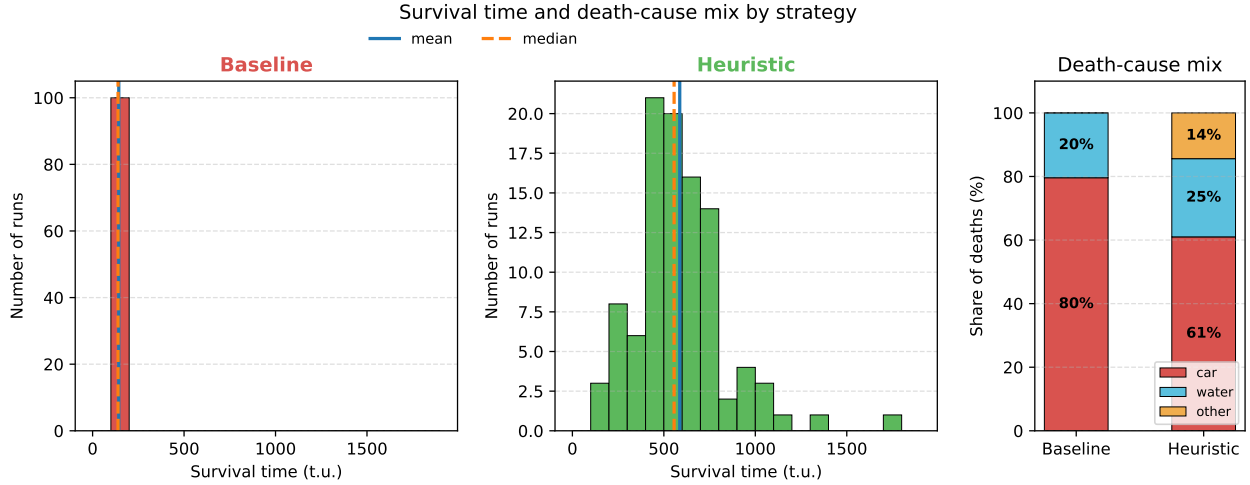


Figure 3: Left and middle: survival time distributions over 100 simulation runs per strategy; solid blue lines mark the mean, dashed orange lines the median. Right: stacked death cause shares. The baseline only experiences car and water deaths, while the heuristic additionally encounters timeout, submerged turtle, and deadly cell deaths.

Among the 28 runs that complete level 1, the mean score is 300 (all ≥ 200 , so the 200 point SMC target is automatically satisfied once level 1 is completed), with about 4.4 deaths (3.4 car + 1.0 water) leaving ≈ 0.6 lives to spare, at a mean completion time of 866 t.u. The binding bottleneck is thus filling the five bays before losing five lives, not reaching 200 points.

The two SMC queries are consistent: Query I estimates a probability of roughly 0.32 for reaching score 200 within 1000 t.u., and Query II finds 28/100 runs completing level 1 within 2000 t.u. Both point to the same qualitative conclusion — the heuristic makes completion possible, while the baseline does not.

5 CONCLUSIONS

We modeled Frogger in Uppaal as a network of timed automata in both a symbolic and a stochastic version. On the symbolic model we verified seven TCTL queries: two safety properties on the environment only system (deadlock freedom and row/offset integrity) and five reachability properties on the full system (reaching specific bays and scores, surviving 100 time units with all lives, and dying by a submerged turtle). On the stochastic extension we ran two SMC queries and compared a baseline random walk against a goal biased safe hop heuristic with identical move weights, so that the contrast isolates the policy. The heuristic raises level 1 completion from 0% to 28% and roughly quadruples survival time, while exposing long game hazards (timeout, submerged turtle, deadly cell) that the baseline never lives long enough to see; the binding constraint on winning is filling five bays before losing five lives, not the 200 point target.

A DESIGN ALTERNATIVES

The final model is the result of several trade offs between verification cost, adherence to the project requirements, and maintainability. We record the main alternatives here so that the rationale behind each final choice is explicit.

A.1 Per object coordinates versus per row offset

The most intuitive way to model movement is to give every object its own mutable `x` and its own clock. We rejected this because the spec makes all objects in a row move in lockstep, so per object clocks are redundant. Instead we store an immutable `(baseX, row)` for each object and mutate a single `row_offset[row]` per lane. The position is derived by wrapping on a ring of 20 cells, `effective_x = (baseX + row_offset[row]) mod VIRTUAL_W`. This collapses many clocks into one per lane and makes lane membership a static fact. The price is a less object oriented layout and the need to handle the off screen buffer globally.

A.2 Dedicated map template vs implicit `getCellType`

We briefly considered a global 2D matrix or a dedicated Map template. We chose an implicit map, namely a pure function of `(x, y)` and row band constants, because it adds no state to the verifier's search. The downside, typical of Uppaal style models, is that a more complex map would force a long if-else chain into `getCellType`.

A.3 Per turtle diving clocks versus a single `DivingCycle`

Giving each diving turtle its own clock would allow diving out of phase, but the project specification requires a shared cycle. A single global `DivingCycle` is therefore both simple and compact. The only limitation is the impossibility of asynchronous diving, which the specification does not require.

A.4 `GameOver` self loop versus the two system toggle

Because `GameOver` has no outgoing edges, query I ($A[] \neg \text{deadlock}$) would fail on the full system. We could have added a trivial self loop to make the terminal location non-blocking and obtained a true one click model. Instead we kept the two commented system declarations, one with only the environment for queries I to II and the other with the full system for queries III to VII. The toggle makes the distinction between environment verification and full system reachability explicit, without introducing an artificial transition whose only purpose is to silence the verifier.

A.5 Visible width wraparound versus the ring of 20 cells

Wrapping modulo the visible width `COLS=18` was the first natural choice, but it ignores the buffer of 2 off screen cells. The spec states that a frog carried offscreen on a floating object re-enters from the opposite side, so the model must treat those 2 extra cells as part of the horizontal ring. We therefore wrap on `VIRTUAL_W = 20 = COLS + 2`. An additional benefit is that `isBetweenTurtleGroup` now uses the same wraparound ring as `effective_x` and the carry logic. This avoids a seam error where column 17 could be treated as adjacent to column 0.

A.6 Local clocks versus a single `globalClock`

Uppaal idiomatically uses local clocks inside templates, but query VI needs to compare wall clock time since game start with the frog's current state. A single `globalClock` is the simplest way to provide that reference. The cost is one extra clock in the state space, while local timeout clocks would not express the same property directly.

REFERENCES

- [1] G. Behrmann, A. David, and K. G. Larsen. *A Tutorial on Uppaal 4.0*. 2006. URL: <https://uppaal.org/texts/new-tutorial.pdf>.
- [2] M. Bernardo and F. Corradini. *Formal Methods for the Design of Real-Time systems*. 2004. URL: <https://link.springer.com/book/10.1007/b110123>.
- [3] A. David et al. *Uppaal SMC Tutorial*. 2018. URL: <https://uppaal.org/texts/uppaal-smc-tutorial.pdf>.
- [4] I. Lee. *UPPAAL tutorial*. 2009. URL: <https://www.seas.upenn.edu/~lee/09cis480/lec-part-4-uppaal-input.pdf>.
- [5] L. Lestingi. “Statistical Model Checking and Uppaal SMC”. 2021. URL: <https://webeep.polimi.it>.
- [6] A. Manini and P. San Pietro. “The Uppaal Tool”. 2005. URL: <https://webeep.polimi.it>.
- [7] P. San Pietro. “Lecture slides of FORMAL METHODS FOR SOFTWARE ENGINEERING course”. 2026. URL: <https://webeep.polimi.it>.
- [8] P. San Pietro and A. Manini. “Hop-paal project specification (Frogger as a Network of Timed Automata)”. 2026. URL: <https://webeep.polimi.it>.
- [9] A. Maggiolo Schettini. *Verifying Real-Time Systems - The Uppaal Model Checker*. 2007. URL: http://groups.di.unipi.it/~maggiolo/Lucidi_TA/VerifyingTA-Uppaal.
- [10] Uppaal Developers. *Uppaal Documentation*. 2026. URL: <https://docs.uppaal.org/>.